



Universidad Nacional Autónoma de México

Practica 2 Compiladores 2027 - 1

Profesora: L. en C.C. Yessica Janeth Pablo Martínez

Ayudante de Laboratorio: Pedro Yamil Salazar González

Alumnos :

Marcial Ahedo Andrick Iorel

Guzmán Ramirez Diego Arturo

Equipo 6

Fecha de entrega : 11 / 09 / 2026

Objetivo

El objetivo de esta práctica fue completar el analizador léxico del lenguaje MiniC a partir de la implementación desarrollada durante la Práctica 1, manteniendo la estructura y las funcionalidades previamente implementadas.

Para completar el lexer se incorporó el reconocimiento de identificadores, palabras reservadas, literales booleanos, operadores compuestos y comentarios de una sola línea. También se conservó el manejo de errores léxicos, permitiendo que el analizador continúe procesando el archivo después de encontrar caracteres inválidos.

Evolución respecto de la Práctica 1

Primero verificamos que la versión heredada compilara correctamente
y que sus 14 pruebas continuaran funcionando. Integramos las pruebas
públicas de la nueva práctica.
Los cambios principales se concentraron en src/lexer/lexer.c. Se reutilizaron el manejo de enteros, símbolos simples, ERROR, TOKEN_EOF, memoria
dinámica y la función advance_position. A partir de esa base se incorporaron funciones para reconocer identificadores, clasificar palabras reservadas, reconocer operadores compuestos mediante anticipación y omitir comentarios actualizando correctamente la posición.

Descripción de la solución

Reconocimiento de identificadores

Se agregaron las funciones auxiliares is_identifier_start() e is_identifier_part().

La primera determina si un carácter puede comenzar un identificador, si corresponde a una letra mayúscula, una letra minúscula o un guion bajo. La segunda determina si un carácter puede continuar formando parte del identificador, permitiendo también la presencia de dígitos.

Cuando el lexer encuentra un carácter inicial válido, consume todos los caracteres consecutivos que pertenecen al patrón y construye un único lexema. Se conserva el uso de almacenamiento dinámico mediante malloc y realloc, de forma similar al reconocimiento de enteros realizado en la Práctica 1.

Palabras reservadas y literales booleanos

Ya que tenemos el identificador completo, se utiliza la función identifier_type() para determinar su tipo.

Esta función compara el lexema con las palabras reservadas, si hay coincidencia, se genera el token correspondiente y de lo contrario se usa IDENTIFIER

Operadores compuestos

Para los operadores de dos caracteres se agregó la función compound_token_type(), encargada de identificar:
== EQUAL
!= NOT_EQUAL
<= LESS_EQUAL
>= GREATER_EQUAL
&& AND
|| OR
Para reconocerlos se utiliza anticipación mediante fgetc(). Cuando el carácter siguiente forma parte de un operador compuesto, ambos caracteres son consumidos y se genera un único token.

Cuando la combinación no corresponde a un operador compuesto, el carácter leído anticipadamente se devuelve al flujo mediante ungetc(), permitiendo que el análisis continúe normalmente.

Esto también permite conservar los operadores simples =, < y >. Por otra parte, los caracteres !, & y | aislados no corresponden a operadores válidos y continúan hasta el mecanismo de generación de ERROR

Comentarios

Para distinguir el operador de división de un comentario, el carácter / se procesa antes de generar directamente un token SLASH.

Si el siguiente carácter también es /, se reconoce el inicio de un comentario y se llama a skip_line_comment(). Esta función consume los caracteres hasta encontrar \n, \r o el final del archivo, sin generar tokens.

Si el segundo carácter no es /, este se devuelve al archivo y el primer / se procesa normalmente como SLASH.

Recuperación de errores
Se mantuvo el mecanismo implementado previamente para generar un token ERROR por cada carácter no reconocido. Después de producirlo, el lexer continúa procesando el siguiente carácter, lo que permite reportar varios errores en una misma entrada.

El análisis siempre finaliza mediante TOKEN_EOF, incluso cuando existen espacios en blanco o comentarios inmediatamente antes del final del archivo.

Resultados y pruebas

Para comprobar la implementación se conservaron los casos utilizados durante la Práctica 1 y se incorporaron las pruebas públicas proporcionadas para la Práctica 2. Además, se diseñaron seis casos adicionales para comprobar situaciones límite y combinaciones no cubiertas directamente por las pruebas públicas.

Agregamos 6 pruebas propias.

Prueba	Qué verifica?
c5_identifier_edges	Casos límite de identificadores, prefijos de palabras reservadas y separación entre enteros e identificadores.
c6_operator_sequences	Combinaciones consecutivas de operadores simples, compuestos e incompletos.
c7_comment_content	Verifica que palabras reservadas, operadores y caracteres inválidos dentro de un comentario sean ignorados.
c8_number_identifier	Comprueba la separación entre números e identificadores cuando aparecen consecutivamente.
c9_p2_error_recovery	Combina errores léxicos con operadores válidos para verificar que el análisis continúe.
c10_comment_isolated_cr	Verifica que un comentario terminado mediante \r aislado finalice correctamente y actualice la posición.

	Casos	Resultado
Pruebas publicas P01	10	10 correctas
Pruebas publicas P02	13	13 correctas
Pruebas propias P01	4	4 correctas
Pruebas propias P02	6	6 correctas

Problemas encontrados

Uno de los principales problemas fue el reconocer operadores compuestos. La implementación heredada de la Práctica 1 procesaba cada símbolo de forma individual, por lo que una entrada como == inicialmente producía dos tokens ASSIGN, != producía un ERROR seguido de ASSIGN.

Para solucionarlo agregamos anticipación de un carácter antes del procesamiento de los símbolos simples. Así determinamos si dos caracteres consecutivos conformaban un operador compuesto. Cuando la combinación no era válida, el segundo carácter se devolvía al flujo mediante ungetc() para ser procesado posteriormente.

Otro problema similar apareció con los comentarios. Debido a que / ya era reconocido como SLASH desde la Práctica 1, //comentario, producía dos tokens SLASH y posteriormente tokens correspondientes al contenido del comentario. La solución consistió en comprobar primero si / estaba seguido de otro /. En ese caso se ignora el contenido utilizando skip_line_comment(); de lo contrario se conserva el comportamiento original y se genera SLASH.

Conclusiones

La Práctica 2 completamos el analizador léxico de MiniC a partir de la practica anterior. Pudimos reutilizar el sistema de tokens, la administración de lexemas, el reconocimiento de enteros y símbolos simples, el seguimiento de posición, los errores léxicos y la generación de TOKEN_EOF.

Las nuevas funcionalidades permitieron reconocer identificadores, palabras reservadas, literales booleanos, operadores compuestos y comentarios de una sola línea. También se incorporó anticipación de caracteres para resolver situaciones en las que diferentes tokens comienzan con el mismo símbolo y se mantuvo la recuperación ante errores sin detener inmediatamente el análisis.

Uso de herramientas de Inteligencia Artificial

Durante el desarrollo de la práctica se utilizó ChatGPT, de OpenAI, como herramienta de apoyo para cumplir todos los requisitos de la práctica, analizar errores de compilación, revisar alternativas de implementación, proponer casos de prueba y apoyar la organización de la documentación
Principalmente lo usamos en la revisión de la redacción para que se entendiera de forma clara el desarrollo de la solución de está práctica, la descripción clara de algunos casos de prueba y sugerencias de nuevos casos aislados para comprobar correctamente el código, para especificar las referencias utilizadas y para organizar los README.md con el formato de markdown y no modificar la plantilla que se encuentra en el ZIP de la práctica.
También la utilizamos para revisar los lineamientos de entrega con el propósito de que todo el contenido corresponda con los requisitos solicitados.

Prompts:
“Revisa el PDF de los lineamientos y dime si nos falta alguno de ellos, te adjunto el reporte y los README.md para que puedas verificarlo, en caso de que falten requisitos dime cuáles.”

“Te adjunto mi PDF para que puedas verificar si me salté alguna sección, sobre todo quiero que mejores la redacción , hazlo de forma formal y de manera que el desarrollo de la solución se entienda con claridad, dime qué parte se puede mejorar para tenerlo en cuenta en el próximo reporte ”

“Agregué las pruebas propias de c6_operator_sequences y c7_comment_content pero no se me ocurren más ¿tienes alguna sugerencia?, dime si necesitas ver mi codigo para revisar que más casos podemos analizar, pensaba alguna prueba que verifique los comentarios aislados pero no tengo de idea de cómo hacerlo, dame sugerencias.”

“Ayudame a redactar estos PDF’S como referencias para mi reporte”

“Te adjunto los README para que puedas escribirlos con el formato de markdown y se vean con claridad todos los ejemplos, sobre todo organizado”

Referencias

1. Salazar González, P. Y. Compiladores — Práctica 1. Facultad de Ciencias, Universidad Nacional Autónoma de México, semestre 2027-1.
2. Salazar González, P. Y. Definición de MiniC. Facultad de Ciencias, Universidad Nacional Autónoma de México, semestre 2027-1.
3. Salazar González, P. Y. Lineamientos de Entrega de Prácticas. Facultad de Ciencias, Universidad Nacional Autónoma de México, semestre 2027-1.
4. Guía para la elaboración del reporte de la Práctica 1. Material proporcionado en el curso de Compiladores, Facultad de Ciencias, UNAM, semestre 2027-1.